

AgroSearch - Protótipo de Motor de Busca Textual

Integrantes: Igor Nunes Moneta; Josenilton Cirne Ramalho Neto; Pedro Henrique Alves Vieira do Nascimento.

1. Objetivo

O AgroSearch foi desenvolvido para simular um motor de busca textual aplicado a documentos técnicos de agricultura. O usuário informa uma consulta e o sistema retorna os documentos mais relevantes por meio de um ranking calculado com TF-IDF. O projeto foi implementado em Python com Streamlit, mantendo o índice invertido e os cálculos de recuperação de informação implementados manualmente, sem uso de TfidfVectorizer ou bibliotecas equivalentes.

2. Pipeline de pré-processamento

Cada documento e a consulta passam pelo mesmo pipeline: (1) conversão para letras minúsculas; (2) remoção de acentos; (3) tokenização com expressão regular; (4) remoção opcional de stopwords; e (5) stemming opcional. A interface possui checkboxes que permitem ativar ou desativar Stopwords e Stemming. Sempre que uma opção é alterada, os documentos, o vocabulário e o índice invertido são reconstruídos.

3. Índice invertido

O índice invertido é uma estrutura do tipo Termo -> Lista de IDs de documentos. Para cada documento são obtidos os tokens únicos e cada termo é associado aos documentos nos quais aparece. Essa estrutura é exibida em tabela na interface, permitindo observar diretamente a relação entre termos e documentos.

4. TF-IDF e ranqueamento

TF: $TF(t,d) = \text{frequência do termo } t \text{ no documento } d / \text{quantidade total de tokens do documento}.$

IDF: $IDF(t) = \log(N / DF(t))$, onde N é o total de documentos e $DF(t)$ é a quantidade de documentos que contêm o termo.

TF-IDF: $TF-IDF(t,d) = TF(t,d) \times IDF(t).$ Para consultas com mais de uma palavra, o sistema soma os valores de TF-IDF de todos os termos da consulta para cada documento. Em seguida, os documentos com pontuação maior que zero são ordenados do maior para o menor score.

5. Interface

Elemento	Função
Checkbox Stopwords	Ativa/desativa a remoção de palavras pouco informativas.
Checkbox Stemming	Ativa/desativa a redução simplificada de sufixos.
Documentos processados	Exibe os tokens após o pré-processamento.
Vocabulário	Exibe todos os termos únicos da coleção.
Índice invertido	Exibe a relação Termo -> Documentos.
Campo de busca	Recebe a consulta do usuário e gera o ranking TF-IDF.

6. Exemplo de execução

Para a consulta “**irrigação soja**”, a aplicação normaliza a consulta para tokens compatíveis com os documentos. Em seguida calcula TF, IDF e TF-IDF para cada termo em cada documento. Documentos que contêm os dois termos tendem a obter pontuação maior do que documentos que contêm apenas um deles. O primeiro colocado é destacado como documento mais relevante.

7. Organização do código

Função	Responsabilidade
remover_acentos()	Normaliza caracteres acentuados.
aplicar_stemmer()	Executa stemming simplificado por remoção de sufixos.
preprocessar()	Executa todo o pipeline textual.
criar_indice_invertido()	Cria a relação termo-documento.
calcular_tf()	Calcula a frequência normalizada do termo.
calcular_idf()	Calcula a raridade do termo na coleção.
buscar()	Soma TF-IDF e cria o ranking final.

8. Testes realizados/sugeridos

soja: Docs 1, 2 e 4; **irrigação:** Docs 1 e 5; **lagartas:** Docs 2 e 4; **milho:** Doc 3. Também deve ser testada uma palavra inexistente, como “trator”, para confirmar que a aplicação retorna a mensagem de ausência de resultados sem gerar erro.

9. Divisão da equipe

Josenilton Cirne Ramalho Neto: responsável pelo pipeline de pré-processamento textual, incluindo tokenização, normalização, remoção de acentos, stopwords, stemming e construção do vocabulário.

Pedro Henrique Alves Vieira do Nascimento: responsável pela construção do índice invertido e pela implementação manual dos cálculos de TF, IDF, TF-IDF e ranqueamento dos documentos.

Igor Nunes Moneta: responsável pela interface em Streamlit, integração das funcionalidades, testes da aplicação e organização da documentação e do relatório.

10. Conclusão

O protótipo atende aos três pilares solicitados: pré-processamento textual configurável, índice invertido e busca ranqueada por TF-IDF. A solução foi mantida propositalmente simples, com foco em demonstrar claramente os conceitos de recuperação de informação e facilitar a explicação do código durante a apresentação.